

BASE DE DATOS II
CONTENIDOS Y ACTIVIDADES

COMANDOS BÁSICOS EN MONGODB

Ahora vamos a ver cuáles son los comandos en MongoDB que nos permitirán realizar las acciones más comunes, esta introducción es importante para que puedas familiarizarte con la sintaxis.

CREANDO LA PRIMERA COLECCIÓN EN MONGODB

Vamos a divertirnos con el CMD para practicar la sintaxis y reforzar el vocabulario específico **creando una colección (base de datos)**:

```
test> use miBaseDeDatosdePrueba
```

Si es correcto MongoDB tendremos la colección creada lista para añadirle documentos

```
test> use miBaseDeDatosdePrueba
switched to db miBaseDeDatosdePrueba
```

Ahora en nuestra colección vamos a **crear el primer documento (registro)**, recordemos que en MongoDB **un documento es el registro en formato BSON, similar a una fila**, escribimos lo siguiente y luego analizaremos la sintaxis de la inserción.

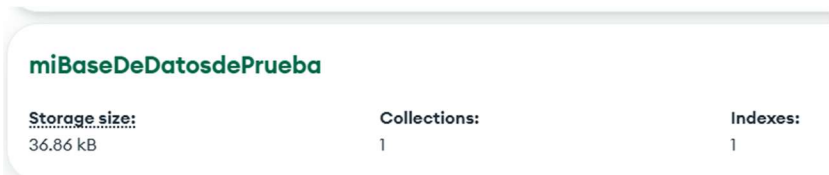
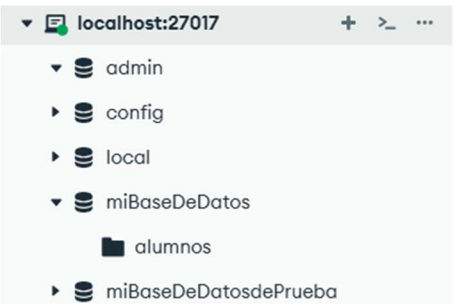
```
db.estudiantes.insertOne({ nombre: "Alvaro", edad: 40, curso: "Base de Datos II" })
```

Si todo fue bien, tendremos el documento (Objet) creado e identificado (Id)

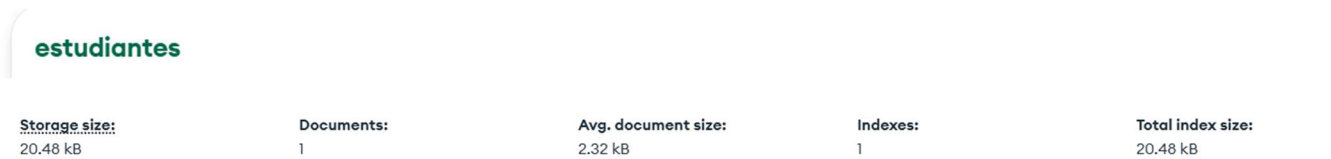
```
acknowledged: true,
insertedId: ObjectId('6842a2b4b288a2cde650eb68')
```

También lo podemos comprobar mirando en el GUI, desde el margen superior derecho presionando el botón actualizar o

Refresh



Como vemos existe la colección **estudiantes**, que contiene 1 **documento**



Ingresando en colección **estudiantes** veremos en detalle la inserción

```
_id: ObjectId('6842a2b4b288a2cde650eb68')
nombre: "Alvaro"
edad: 40
curso: "Base de Datos II"
```

Sencillo ¿No? Probemos más comandos...

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

INSERTAR VARIOS DOCUMENTOS EN MONGODB

Continuemos con el CMD **añadiendo varios documentos a nuestra colección (base de datos)**:

```
miBaseDeDatosdePrueba> db.estudiantes.insertMany([
...   { nombre: "Carlos", edad: 22, curso: "Bases de Datos" },
...   { nombre: "María", edad: 21, curso: "Bases de Datos II" }
... ])
```

Si la operación fue correcta

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6842a913b288a2cde650eb69'),
    '1': ObjectId('6842a913b288a2cde650eb6a')
  }
}
```

Pero....

```
SyntaxError: Unexpected token, expected ",", (4:0)
```

El punto donde se encuentra lo veremos claramente identificado

```
> 4 | db.estudiantes.insertOne({
    ^
```

Corregido el error o correctamente procesada la primera acción, refrescando nuestra colección ya deberíamos ver desde los nuevos documentos desde el GUI

```
_id: ObjectId('6842a913b288a2cde650eb69')
nombre : "Carlos"
edad : 22
curso : "Bases de Datos"
```

```
_id: ObjectId('6842a913b288a2cde650eb6a')
nombre : "María"
edad : 21
curso : "Bases de Datos II"
```

O desde el CMD si te gusta escribir comandos, podemos decirle a MongoDB que nos muestre todos los documentos de la colección:

```
miBaseDeDatosdePrueba> db.estudiantes.find()
```

Así veremos la colección completa

```
{
  "_id": ObjectId('6842a2b4b288a2cde650eb68'),
  "nombre": 'Alvaro',
  "edad": 40,
  "curso": 'Base de Datos II'
},
{
  "_id": ObjectId('6842a913b288a2cde650eb69'),
  "nombre": 'Carlos',
  "edad": 22,
  "curso": 'Bases de Datos'
},
{
  "_id": ObjectId('6842a913b288a2cde650eb6a'),
  "nombre": 'María',
  "edad": 21,
  "curso": 'Bases de Datos II'
}
```

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

TRABAJANDO CON DOCUMENTOS EN MONGODB

Lo que hicimos anteriormente fue una operación de **lectura (búsqueda)** utilizando el comando **find()**, pero podemos hacer muchas más, por ejemplo, **buscar por alguna condición específica**, como una edad específica.

```
miBaseDeDatosdePrueba> db.estudiantes.find({ edad: 22 })|
```

Si es correcto y existe algún documento veremos:

```
miBaseDeDatosdePrueba> db.estudiantes.find({ edad: 22 })
[
  {
    _id: ObjectId('6842a913b288a2cde650eb69'),
    nombre: 'Carlos',
    edad: 22,
    curso: 'Bases de Datos'
  }
]
```

Pero si el documento no existe, por ejemplo, la edad consultada fuera 23 años:

```
miBaseDeDatosdePrueba> db.estudiantes.find({ edad: 23 })
```

Veremos...

```
miBaseDeDatosdePrueba> db.estudiantes.find({ edad: 23 })
miBaseDeDatosdePrueba> |
```

Es decir, MongoDB nos brinda la posibilidad que volvamos a consultar, ya que, en la colección, no encontró algún documento con la condición específica.

¿Puedo complejizar la búsqueda? Si, por ejemplo, si necesitara conocer concretamente quienes son todos los estudiantes del curso de Bases de Datos II cuya edad sea **Mayor Que** 22 años, para ello necesito utilizar **operadores**, en este caso utilizaré el **operador \$gt** que significa “**Greater Than**” (**Mayor Que**):

```
miBaseDeDatosdePrueba> db.estudiantes.find({ edad: { $gt: 22 } })
[
  {
    _id: ObjectId('6842a2b4b288a2cde650eb68'),
    nombre: 'Alvaro',
    edad: 40,
    curso: 'Base de Datos II'
  }
]
```

Si es correcto:

```
{
  _id: ObjectId('6842a2b4b288a2cde650eb68'),
  nombre: 'Alvaro',
  edad: 40,
  curso: 'Base de Datos II'
}
```

Pero... puede no serlo

```
miBaseDeDatosdePrueba> db.estudiantes.find({ edad: { $gt: 40 } })
```

Y el resultado ya lo conocemos, MongoDB es de pocas palabras.

```
miBaseDeDatosdePrueba> db.estudiantes.find({ edad: { $gt: 40 } })
miBaseDeDatosdePrueba> |
```

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

ACTUALIZANDO DOCUMENTOS EN MONGODB

Para actualizar un documento es fundamental que exista, aquí modificaremos en la colección estudiantes la edad de Carlos a 23 años:

```
db.estudiantes.updateOne({ nombre: "Carlos" }, { $set: { edad: 23 } })
```

Si es correcto, veremos que se actualizó UN 1 valor (matchedCount):

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Si es incorrecto y no existe el documento.

```
db.estudiantes.updateOne({ nombre: "Lucía" }, { $set: { edad: 23 } })
```

MongoDB nos mostrará:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Y es correcto, porque no tenemos a Lucia registrada.

ELIMINAR DOCUMENTOS EN MONGODB

Para realizar el borrado de un documento le diremos al Shell de MongoDB

```
db.estudiantes.deleteOne({ nombre: "Carlos" })|
```

Si es correcto, el documento se habrá borrado.

```
{ acknowledged: true, deletedCount: 1 }
```

¿Cómo lo comprobamos? Utilizando nuevamente el comando **find()**

```
miBaseDeDatosdePrueba> db.estudiantes.find()
```

Y el resultado será:

```
{
  _id: ObjectId('6842a2b4b288a2cde650eb68'),
  nombre: 'Alvaro',
  edad: 40,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842a913b288a2cde650eb6a'),
  nombre: 'María',
  edad: 21,
  curso: 'Bases de Datos II'
}
```

Carlos ya no es un documento en nuestra colección en MongoDB.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

ALGUNOS COMANDOS OPERATIVOS EN MONGODB

Pero antes de avanzar, te voy a dejar algunos comandos muy útiles, ideales cuando comenzamos a utilizar con mayor profundidad **MongoDb** por Shell o GUI.

¡ATENCIÓN!

Existen diferencias en la sintaxis entre **mongosh** y **MongoDB Compass** para realizar las consultas, en **MongoDB Compass**, solo escribimos el filtro en BSON:



Si la consulta es correcta sintácticamente el botón **Find** se activa



Y el resultado será:

```
_id: ObjectId('68424395b288a2cde650eb67')
nombre: "Alvaro"
edad: 35
curso: "Base de Datos II"
```

```
_id: ObjectId('6842a2b4b288a2cde650eb68')
nombre: "Alvaro"
edad: 40
curso: "Base de Datos II"
```

Si bien es cierto que MongoDB es un gestor de pocas palabras, **mongosh** requiere que escribamos la instrucción completa para obtener una respuesta:

```
db.estudiantes.find({ curso: "Base de Datos II" });
```

Si es correcto y existen documentos, el resultado será:

```
{
  _id: ObjectId('68424395b288a2cde650eb67'),
  nombre: 'Alvaro',
  edad: 35,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842a2b4b288a2cde650eb68'),
  nombre: 'Alvaro',
  edad: 40,
  curso: 'Base de Datos II'
}
```

¿Querés ver que pasa si somos sintéticos en el modo Shell? Probemos:

```
miBaseDeDatosdePrueba> { "curso": "Base de Datos II" }
```

Y luego de algún tiempo...

```
Uncaught:
SyntaxError: Missing semicolon. (1:9)

> 1 | { "curso": "Base de Datos II" }
    |      ^
    |
    | 2 |
    | 3 | db.estudiantes.find({ curso: "Base de Datos II" });
    | 4 |
```

Simple, no entendió la escritura y espera en este caso un punto y coma (semicolon), perdido según el Shell.

¿Continuamos?

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

BUSCAR POR CONDICIÓN

Sabiendo ya la sintaxis, ahora vamos a buscar por ejemplo con la condición del nombre del curso específico de "Base de Datos II":

```
db.estudiantes.find({ curso: "Base de Datos II" });
```

El resultado ya lo vimos en el ejemplo que nos muestra la sintaxis,

```
{
  _id: ObjectId('68424395b288a2cde650eb67'),
  nombre: 'Alvaro',
  edad: 35,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842a2b4b288a2cde650eb68'),
  nombre: 'Alvaro',
  edad: 40,
  curso: 'Base de Datos II'
}
```

BUSCAR ENTRE DOS CONDICIONES

Hasta este momento nuestra colección tiene solamente dos documentos, primeramente voy a insertar algunos documentos nuevos:

```
miBaseDeDatosdePrueba> db.estudiantes.insertMany([
...   { nombre: "María", edad: 19, curso: "Bases de Datos II" },
...   { nombre: "Juana", edad: 24, curso: "Bases de Datos II" },
...   { nombre: "Andrea", edad: 26, curso: "Bases de Datos II" },
... ])
{
```

Si la operación fue correcta, tendremos 3 nuevos documentos para trabajar:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6842d6a9b288a2cde650eb6b'),
    '1': ObjectId('6842d6a9b288a2cde650eb6c'),
    '2': ObjectId('6842d6a9b288a2cde650eb6d')
  }
}
```

¿CÓMO BUSCAMOS CON VARIOS CONDICIONALES?

Por ejemplo, vamos a buscar estudiantes con edad entre 18 y 25:

```
db.estudiantes.find({ edad: { $gte: 18, $lte: 25 } })
```

Si es correcto, veremos de nuestra colección:

```
[
  {
    _id: ObjectId('6842a913b288a2cde650eb6a'),
    nombre: 'María',
    edad: 21,
    curso: 'Bases de Datos II'
  },
  {
    _id: ObjectId('6842d6a9b288a2cde650eb6b'),
    nombre: 'María',
    edad: 19,
    curso: 'Bases de Datos II'
  },
  {
    _id: ObjectId('6842d6a9b288a2cde650eb6c'),
    nombre: 'Juana',
    edad: 24,
    curso: 'Bases de Datos II'
  }
]
```

BASE DE DATOS II
CONTENIDOS Y ACTIVIDADES

MUCHOS OTROS COMANDOS INTERESANTES

Buscar estudiantes de Bases de Datos II que sean mayores de 20 años:

```
db.estudiantes.find({ curso: "Bases de Datos II", edad: { $gt: 20 } })
```

Y.. el Shell de MongoDB nos responde:

```
[
  {
    _id: ObjectId('6842a913b288a2cde650eb6a'),
    nombre: 'Maria',
    edad: 21,
    curso: 'Bases de Datos II'
  },
  {
    _id: ObjectId('6842d6a9b288a2cde650eb6c'),
    nombre: 'Juana',
    edad: 24,
    curso: 'Bases de Datos II'
  },
  {
    _id: ObjectId('6842d6a9b288a2cde650eb6d'),
    nombre: 'Andrea',
    edad: 26,
    curso: 'Bases de Datos II'
  }
]
```

Buscar estudiantes que cursen Base de Datos y Base de Datos II, por si no lo habías notado, en uno de los primeros ejemplos creamos en nuestra colección ambas asignaturas y le añadimos documentos, pero en una operación de borrado, eliminamos el documento Carlos, lo terrible aquí es que junto con el estudiante se fue la asignatura Base de Datos por completo, MongoDB simplemente ejecuto nuestra orden, ¿Se acuerdan de la integridad referencial de SQL? Nos podría haber salvado de la pérdida total de lo que sería una tabla: Probemos.

```
db.estudiantes.find({ curso: { $in: ["Base de Datos", "Base de Datos II"] } })
```

¿Qué resultado obtuvimos?

```
[
  {
    _id: ObjectId('68424395b288a2cde650eb67'),
    nombre: 'Alvaro',
    edad: 35,
    curso: 'Base de Datos II'
  },
  {
    _id: ObjectId('6842a2b4b288a2cde650eb68'),
    nombre: 'Alvaro',
    edad: 40,
    curso: 'Base de Datos II'
  }
]
```

¿Es correcto? Si, MongoDB nos dice que tenemos 2 documentos en la asignatura Base de Datos II, de Base de Datos ya sabemos que eliminamos el único registro existente, pero... ¿Qué pasó con los nuevos ingresos que hicimos? ¿Falló MongoDB? No, los nuevos documentos se registraron para Bases de Datos II, la letra **s**, hace la diferencia. Probamos incluyéndola en la consulta.

```
db.estudiantes.find({ curso: { $in: ["Base de Datos", "Base de Datos II", "Bases de Datos II"] } })
```

¿Y ahora?

```
{
  _id: ObjectId('68424395b288a2cde650eb67'),
  nombre: 'Alvaro',
  edad: 35,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842a2b4b288a2cde650eb68'),
  nombre: 'Alvaro',
  edad: 40,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842a913b288a2cde650eb6a'),
  nombre: 'Maria',
  edad: 21,
  curso: 'Bases de Datos II'
},
{
  _id: ObjectId('6842d6a9b288a2cde650eb6b'),
  nombre: 'Maria',
  edad: 19,
  curso: 'Bases de Datos II'
},
{
  _id: ObjectId('6842d6a9b288a2cde650eb6c'),
  nombre: 'Juana',
  edad: 24,
  curso: 'Bases de Datos II'
},
{
  _id: ObjectId('6842d6a9b288a2cde650eb6d'),
  nombre: 'Andrea',
  edad: 26,
  curso: 'Bases de Datos II'
}
```

BASE DE DATOS II
CONTENIDOS Y ACTIVIDADES

OPERADORES COMUNES EN MONGODB

En la siguiente tabla te dejo los operadores más utilizados, con ellos un ejemplo de la sintaxis de combinación para DBCompass.

OPERADOR	SIGNIFICADO	DBCOMPASS
\$eq	Igual a	{ curso: { \$eq: "Base de Datos II" } }
\$ne	Distinto de	{ curso: { \$ne: "Bases de Datos II" } }
\$gt	Mayor que	{ edad: { \$gt: 20 } }
\$gte	Mayor o igual	{ edad: { \$gte: 18 } }
\$lt	Menor que	{ edad: { \$lt: 25 } }
\$lte	Menor o igual	{ edad: { \$lte: 30 } }
\$in	Dentro de un conjunto	{ curso: { \$in: ["Base de Datos II", "Bases de Datos II"] } }
\$nin	No está en un conjunto	{ curso: { \$nin: ["Base de Datos", "Base de Datos II"] } }
\$and	Y lógico	{ \$and: [{ edad: { \$gt: 18 } }, { curso: "Bases de Datos II" }] }
\$or	O lógico	{ \$or: [{ edad: { \$lt: 18 } }, { curso: "Bases de Datos II" }] }
\$not	Negación	{ edad: { \$not: { \$gt: 20 } } }
\$exists	El campo existe o no	{ telefono: { \$exists: true } }
\$regex	Coincidencia por patrón	{ nombre: { \$regex: "^L" } } (nombres que empiezan con "L")

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

COMBINACIONES

Por ejemplo, podemos buscar en nuestra colección a todos los estudiantes entre 18 y 25 años que cursan Base de Datos II.

```
miBaseDeDatosdePrueba> db.estudiantes.find({
...   edad: { $gte: 18, $lte: 25 },
...   curso: "Bases de Datos II"
... })
```

y... MongoDB Shell nos dice:

```
{
  _id: ObjectId('6842a913b288a2cde650eb6a'),
  nombre: 'María',
  edad: 21,
  curso: 'Bases de Datos II'
},
{
  _id: ObjectId('6842d6a9b288a2cde650eb6b'),
  nombre: 'María',
  edad: 19,
  curso: 'Bases de Datos II'
},
{
  _id: ObjectId('6842d6a9b288a2cde650eb6c'),
  nombre: 'Juana',
  edad: 24,
  curso: 'Bases de Datos II'
}
]
```

O saber quiénes son los estudiantes de Base de Datos II mayores de 30 años:

```
miBaseDeDatosdePrueba> db.estudiantes.find({
...   $or: [
...     { curso: "Base de Datos II" },
...     { edad: { $gt: 30 } }
...   ]
... })
[
```

El resultado será:

```
{
  _id: ObjectId('68424395b288a2cde650eb67'),
  nombre: 'Alvaro',
  edad: 35,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842a2b4b288a2cde650eb68'),
  nombre: 'Alvaro',
  edad: 40,
  curso: 'Base de Datos II'
}
]
```

Podríamos buscar estudiantes cuyos nombres comiencen con la letra A:

```
miBaseDeDatosdePrueba> db.estudiantes.find({
...   nombre: { $regex: "^A" }
... })
```

Y entonces:

```
{
  _id: ObjectId('68424395b288a2cde650eb67'),
  nombre: 'Alvaro',
  edad: 35,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842a2b4b288a2cde650eb68'),
  nombre: 'Alvaro',
  edad: 40,
  curso: 'Base de Datos II'
},
{
  _id: ObjectId('6842d6a9b288a2cde650eb6d'),
  nombre: 'Andrea',
  edad: 26,
  curso: 'Bases de Datos II'
}
]
```

Es todo, ahora estás en condiciones de comenzar a trabajar.